
JAVA

DE 0 A 100

La guía completa, en español, para aprender Java desde la primera línea de código hasta conceptos de nivel universitario

Fundamentos · Programación Orientada a Objetos · Contexto y ámbito en Java
Excepciones · Colecciones · Lambdas y Streams · Concurrencia

Guía preparada para un aprendizaje progresivo, con ejemplos claros y comentados

Índice

Capítulo 1. ¿Qué es Java y cómo funciona?	7
La idea central: "escribe una vez, ejecuta en cualquier lugar"	7
El camino de un programa Java	7
¿Por qué Java es tan usado?	7
Capítulo 2. Tu primer programa: Hola Mundo	8
Compilar y ejecutar desde la terminal	8
Los comentarios	8
Capítulo 3. Variables y tipos de datos	10
Tipos primitivos	10
Tipos de referencia	10
Constantes con final	10
Conversión de tipos (casting)	10
Capítulo 4. Operadores	12
Operadores aritméticos	12
Operadores relacionales y lógicos	12
Asignación, incremento y decremento	12
Precedencia de operadores	12
Capítulo 5. Estructuras de control condicionales	13
if / else if / else	13
Operador ternario	13
switch clásico	13
switch expression (Java moderno)	14
Capítulo 6. Bucles: repitiendo instrucciones	15
Bucle for	15
Bucle while	15
Bucle do-while	15
for-each: recorrer colecciones	15
break y continue	15
Bucles anidados	16
Capítulo 7. Arrays (arreglos)	17
Declaración e inicialización	17
Recorrer un array	17
Arrays multidimensionales	17

Arrays de objetos	18
Capítulo 8. Cadenas de texto (String)	19
Inmutabilidad: la clave para entender String	19
Métodos comunes de String	19
Comparar cadenas: == vs equals()	19
StringBuilder: para construir texto eficientemente	19
Capítulo 9. Métodos: organizando el código en bloques reutilizables	21
Anatomía de un método	21
Métodos sin valor de retorno: void	21
Sobrecarga de métodos (overloading)	21
Paso de parámetros por valor	22
Varargs: número variable de argumentos	22
Recursividad	22
Capítulo 10. Programación Orientada a Objetos (POO): clases y objetos	24
Clase vs objeto	24
Constructores	24
La palabra clave this	25
Constructor por defecto	25
Capítulo 11. El contexto en Java: ámbito, this y static	26
11.1 Ámbito (scope) de las variables	26
11.2 Contexto estático vs contexto de instancia	26
11.3 El significado contextual de this	27
11.4 Contexto en clases anidadas y lambdas	27
Capítulo 12. Encapsulamiento y modificadores de acceso	29
Modificadores de acceso	29
Getters y setters	29
Capítulo 13. Herencia	30
La palabra clave super	30
Sobrescritura de métodos (override)	30
Capítulo 14. Polimorfismo	32

Upcasting, downcasting e instanceof	32
Capítulo 15. Clases abstractas e interfaces	33
Clases abstractas	33
Interfaces	33
Métodos default y static en interfaces	34
Capítulo 16. Manejo de excepciones	35
try / catch / finally	35
Excepciones checked vs unchecked	35
try-with-resources	35
Crear excepciones propias	35
Capítulo 17. Colecciones: listas, conjuntos y mapas	37
ArrayList: una lista dinámica	37
HashSet: elementos sin duplicados	37
HashMap: pares clave-valor	37
Capítulo 18. Genéricos	39
Una clase genérica propia	39
Métodos genéricos	39
Capítulo 19. Programación funcional: lambdas y Streams	40
Interfaces funcionales	40
Expresiones lambda	40
Streams: procesar colecciones de forma declarativa	40
Capítulo 20. Entrada/salida (I/O) y archivos	42
Leer datos desde el teclado	42
Escribir y leer archivos de texto	42
Capítulo 21. Concurrencia básica: hilos (Threads)	43
Crear un hilo con Runnable	43
El contexto de un hilo	43
Capítulo 22. Buenas prácticas y hacia dónde seguir	44
Convenciones de nombres	44
Javadoc: documentar tu código	44

Pruebas automatizadas con JUnit (introducción) 44

Herramientas de construcción: Maven y Gradle 44

¿Qué seguir aprendiendo? 44

Cómo usar esta guía

Este libro está pensado para que puedas empezar sin ningún conocimiento previo de programación y llegar, capítulo a capítulo, hasta temas que normalmente se ven en una carrera universitaria. Cada capítulo se apoya en el anterior, así que se recomienda leerlos en orden la primera vez.

A lo largo del texto encontrarás tres tipos de recuadros:

■ **NOTA:** Explicaciones adicionales o datos curiosos que complementan el tema.

■ **CONSEJO:** Recomendaciones prácticas para escribir mejor código.

■■ **CUIDADO:** Errores comunes que debes evitar como principiante.

Todos los ejemplos de código están completos y comentados en español. Se recomienda no solo leerlos, sino escribirlos y ejecutarlos tú mismo: la programación se aprende programando, no solo leyendo.

Un capítulo especial —el número 11— está dedicado por completo a explicar el **contexto en Java**: cómo funciona el ámbito de las variables, la diferencia entre el contexto estático y el contexto de instancia, y el significado de `this` según dónde se use. Es uno de los temas que más dudas genera, así que se aborda con calma y con varios ejemplos.

Capítulo 1. ¿Qué es Java y cómo funciona?

Java es un lenguaje de programación creado en 1995 por Sun Microsystems (hoy propiedad de Oracle). Se usa para crear aplicaciones de escritorio, aplicaciones web, sistemas empresariales, aplicaciones Android y muchísimo software que corre en servidores. Si es tu primer contacto con la programación, no te preocupes: en este libro empezamos desde cero, como si estuvieras en tu primer día de clase, y llegamos poco a poco hasta temas de nivel universitario.

La idea central: "escribe una vez, ejecuta en cualquier lugar"

La gran promesa de Java es que un mismo programa puede ejecutarse en Windows, macOS o Linux sin cambiar el código. Esto es posible gracias a tres piezas que trabajan juntas:

- **JDK (Java Development Kit):** el "kit de herramientas" que instalas para programar. Incluye el compilador y todo lo necesario para escribir y construir programas.
- **JRE (Java Runtime Environment):** el entorno necesario para *ejecutar* programas Java ya compilados (sin él no puedes programar, solo correr programas).
- **JVM (Java Virtual Machine):** una "máquina virtual" que interpreta el código compilado y lo traduce a instrucciones que tu sistema operativo entiende.

■ **NOTA:** Cuando programas en Java, tu código fuente (.java) se compila a un lenguaje intermedio llamado *bytecode* (.class). Ese bytecode no es directamente instrucciones de tu procesador: es la JVM quien lo lee y lo ejecuta. Por eso el mismo archivo .class funciona igual en cualquier sistema operativo que tenga una JVM instalada.

El camino de un programa Java

- 1 Escribes tu código en un archivo con extensión .java (por ejemplo `Hola.java`).
- 2 El compilador `javac` traduce ese archivo a bytecode, generando un archivo `Hola.class`.
- 3 El comando `java` le pide a la JVM que cargue ese .class y lo ejecute, línea por línea, dentro de su propio entorno.

¿Por qué Java es tan usado?

- Es **orientado a objetos**: organiza el código en "objetos" que representan cosas del mundo real (un carro, un usuario, una cuenta bancaria).
- Tiene **tipado fuerte**: el compilador revisa muchos errores antes de que el programa se ejecute, lo cual evita sorpresas.
- Cuenta con una **enorme comunidad** y bibliotecas para casi cualquier tarea.
- Se usa en bancos, aerolíneas, tiendas en línea, apps Android y sistemas académicos.

Capítulo 2. Tu primer programa: Hola Mundo

Toda persona que aprende a programar empieza con el mismo ritual: hacer que la computadora salude. Vamos a entender cada línea de este programa como si fuera la primera vez que ves código en tu vida.

Archivo: *Hola.java*

```
public class Hola {  
    public static void main(String[] args) {  
        System.out.println("¡Hola, mundo!");  
    }  
}
```

Explicación línea por línea

- `public class Hola`: en Java, todo el código vive dentro de una **clase**. Aquí declaramos una clase llamada *Hola*. El nombre del archivo debe coincidir exactamente con el nombre de la clase pública.
- `public static void main(String[] args)`: este es el **método principal**. Es el punto de entrada: la JVM siempre busca este método exacto para saber por dónde empezar a ejecutar el programa.
- `System.out.println(...)`: imprime un texto en la consola, seguido de un salto de línea.
- Cada instrucción termina en `;` (punto y coma), y los bloques de código se agrupan entre llaves `{ }`.

Compilar y ejecutar desde la terminal

Terminal / línea de comandos

```
# 1. Compilar (genera Hola.class)  
javac Hola.java  
  
# 2. Ejecutar  
java Hola  
  
# Salida esperada:  
¡Hola, mundo!
```

Los comentarios

Los comentarios son texto que el compilador ignora por completo; sirven para que tú (o quien lea tu código) entienda qué está pasando.


```
// Esto es un comentario de una sola línea

/* Esto es un comentario
   de varias líneas */

/**
 * Esto es un comentario Javadoc,
 * usado para documentar clases y métodos.
 */
```

■ **CONSEJO:** No necesitas instalar nada complicado para empezar: basta con el JDK (descargable gratis desde Oracle o distribuciones como Eclipse Temurin) y un editor de texto. Herramientas como IntelliJ IDEA, Eclipse o VS Code con la extensión de Java hacen el proceso más cómodo, pero no son obligatorias.

Capítulo 3. Variables y tipos de datos

Una **variable** es un espacio con nombre donde guardamos un valor para usarlo más adelante. En Java, cada variable tiene un **tipo** fijo, que le dice al compilador qué clase de datos puede guardar y cuánta memoria reservar.

Tipos primitivos

Java tiene 8 tipos primitivos, que representan valores simples (no objetos):

- `int`: números enteros (ej. 10, -42). El más usado.
- `long`: enteros muy grandes (se escribe con sufijo L: 10000000000L).
- `double`: números decimales de doble precisión (ej. 3.1416).
- `float`: decimales de precisión simple (sufijo f: 3.14f).
- `char`: un solo carácter, entre comillas simples ('A').
- `boolean`: verdadero o falso (true / false).
- `byte` y `short`: enteros pequeños, útiles para ahorrar memoria en casos específicos.

```
int edad = 25;
double precio = 19.99;
char inicial = 'M';
boolean esMayor = true;
long poblacionMundial = 8_100_000_000L;
```

Tipos de referencia

Además de los primitivos, existen tipos de referencia: **objetos** como `String` (texto), arrays, y cualquier clase que tú mismo definas. A diferencia de los primitivos, una variable de referencia no guarda el valor directamente: guarda una "dirección" que apunta al objeto en memoria.

```
String nombre = "Ana";
int[] numeros = {1, 2, 3};
```

Constantes con final

Cuando un valor no debe cambiar nunca, se declara con la palabra clave `final`. Por convención, las constantes se escriben en mayúsculas.

```
final double IVA = 0.12;
// IVA = 0.20; // Esto daría error de compilación
```

Conversión de tipos (casting)

A veces necesitas convertir un tipo en otro. Cuando la conversión es "segura" (por ejemplo, de `int` a `double`), Java lo hace automáticamente. Cuando puede perder información (de `double` a `int`), debes pedirlo explícitamente.

```
int entero = 10;
double decimal = entero;      // conversión automática (ensanchamiento)

double alto = 9.7;
int redondeado = (int) alto;  // conversión explícita: redondeado = 9
```

■■ **CUIDADO:** Al convertir un `double` a `int` con casting, Java **trunca** la parte decimal (no redondea). 9.7 se convierte en 9, no en 10.

Capítulo 4. Operadores

Los operadores son símbolos que le indican a Java qué operación realizar sobre uno o más valores (llamados operandos).

Operadores aritméticos

```
int a = 10, b = 3;
System.out.println(a + b); // 13 (suma)
System.out.println(a - b); // 7 (resta)
System.out.println(a * b); // 30 (multiplicación)
System.out.println(a / b); // 3 (división entera, se descarta el decimal)
System.out.println(a % b); // 1 (módulo: el residuo de la división)
```

■ ■ **CUIDADO:** Si divides dos `int`, el resultado es otro `int` (sin decimales). Para obtener un resultado decimal, al menos uno de los operandos debe ser `double`: `10.0 / 3` da 3.333...

Operadores relacionales y lógicos

- Relacionales: `==`, `!=`, `>`, `<`, `>=`, `<=` — comparan valores y producen un `boolean`.
- Lógicos: `&&` (Y), `||` (O), `!` (negación) — combinan condiciones booleanas.

```
int edad = 20;
boolean tieneCarnet = true;
boolean puedeConducir = (edad >= 18) && tieneCarnet; // true
```

Asignación, incremento y decremento

```
int x = 5;
x += 3; // equivale a x = x + 3; -> x vale 8
x -= 2; // x vale 6
x *= 2; // x vale 12
x++; // incrementa en 1 -> x vale 13
x--; // decrementa en 1 -> x vale 12
```

Precedencia de operadores

Igual que en matemáticas, la multiplicación y división se evalúan antes que la suma y la resta. Cuando tengas dudas, usa paréntesis: nunca hacen daño y hacen tu código más fácil de leer.

```
int resultado = 2 + 3 * 4; // 14, no 20
int resultadoClaro = 2 + (3 * 4); // 14, más explícito
```

Capítulo 5. Estructuras de control condicionales

Las condicionales permiten que tu programa tome decisiones: ejecutar un bloque de código u otro dependiendo de si una condición es verdadera o falsa.

if / else if / else

```
int nota = 85;

if (nota >= 90) {
    System.out.println("Excelente");
} else if (nota >= 70) {
    System.out.println("Aprobado");
} else {
    System.out.println("Reprobado");
}
// Salida: Aprobado
```

Operador ternario

Es una forma compacta de escribir un if/else simple que devuelve un valor.

```
int edad = 16;
String estado = (edad >= 18) ? "Adulto" : "Menor de edad";
System.out.println(estado); // Menor de edad
```

switch clásico

```
int dia = 3;
switch (dia) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    case 3:
        System.out.println("Miércoles");
        break;
    default:
        System.out.println("Otro día");
}
```

■■ **CUIDADO:** Si olvidas el `break`, la ejecución "cae" al siguiente `case` aunque no se cumpla la condición. A esto se le llama *fall-through* y es una fuente común de errores.

switch expression (Java moderno)

Desde Java 14, existe una forma más moderna y segura de escribir un switch, usando flechas -> que evitan el problema del fall-through y permiten devolver un valor directamente.

```
int dia = 3;
String nombreDia = switch (dia) {
    case 1 -> "Lunes";
    case 2 -> "Martes";
    case 3 -> "Miércoles";
    default -> "Otro día";
};
System.out.println(nombreDia); // Miércoles
```

Capítulo 6. Bucles: repitiendo instrucciones

Los bucles (o ciclos) permiten repetir un bloque de código varias veces sin tener que escribirlo una y otra vez.

Bucle for

Ideal cuando sabes exactamente cuántas veces quieres repetir algo.

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("Vuelta número " + i);  
}  
// Imprime del 1 al 5
```

El for tiene tres partes separadas por punto y coma: inicialización (`int i = 1`), condición (`i <= 5`) e incremento (`i++`).

Bucle while

Se repite mientras la condición sea verdadera. Útil cuando no sabes de antemano cuántas vueltas necesitas.

```
int contador = 0;  
while (contador < 3) {  
    System.out.println("Contador: " + contador);  
    contador++;  
}
```

Bucle do-while

Se garantiza que el bloque se ejecute **al menos una vez**, porque la condición se evalúa al final.

```
int numero;  
do {  
    numero = 7;  
    System.out.println("El número es " + numero);  
} while (numero != 7);
```

for-each: recorrer colecciones

```
int[] edades = {18, 25, 30};  
for (int edad : edades) {  
    System.out.println(edad);  
}
```

break y continue

- **break:** termina el bucle inmediatamente.
- **continue:** salta a la siguiente iteración, sin ejecutar el resto del cuerpo del bucle en la vuelta actual.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;           // se detiene por completo al llegar a 5  
    if (i % 2 == 0) continue;   // se salta los números pares  
    System.out.println(i);  
}  
// Imprime: 1, 3
```

Bucles anidados

Un bucle dentro de otro es muy común, por ejemplo para recorrer una tabla o matriz.

```
for (int fila = 1; fila <= 3; fila++) {  
    for (int columna = 1; columna <= 3; columna++) {  
        System.out.print(fila * columna + " ");  
    }  
    System.out.println();  
}
```


Capítulo 7. Arrays (arreglos)

Un array es una estructura que guarda varios valores del mismo tipo, uno detrás de otro, accesibles mediante un índice numérico que siempre empieza en 0.

Declaración e inicialización

```
int[] numeros = new int[5];    // array de 5 enteros, todos en 0 por defecto
numeros[0] = 10;
numeros[1] = 20;

int[] otro = {1, 2, 3, 4, 5};  // inicialización directa con valores

String[] nombres = {"Ana", "Luis", "Eva"};
```

■ ■ **CUIDADO:** El primer elemento de un array está en el índice 0, no en el 1. Si un array tiene 5 elementos, sus índices válidos van de 0 a 4. Acceder al índice 5 lanza un error (`ArrayIndexOutOfBoundsException`).

Recorrer un array

```
int[] numeros = {5, 10, 15, 20};

// Con for clásico (tienes acceso al índice)
for (int i = 0; i < numeros.length; i++) {
    System.out.println("Índice " + i + ": " + numeros[i]);
}

// Con for-each (más simple si no necesitas el índice)
for (int n : numeros) {
    System.out.println(n);
}
```

Arrays multidimensionales

Un array de arrays, útil para representar tablas o matrices.

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

System.out.println(matriz[1][2]); // fila 1, columna 2 -> imprime 6

for (int[] fila : matriz) {
    for (int valor : fila) {
        System.out.print(valor + " ");
    }
    System.out.println();
}
```

Arrays de objetos

También puedes crear arrays de tipos de referencia, como String o clases propias que veremos más adelante.

```
String[] frutas = new String[3];
frutas[0] = "Manzana";
frutas[1] = "Pera";
frutas[2] = "Uva";
```

Capítulo 8. Cadenas de texto (String)

Un String representa texto. Aunque se usa como si fuera un tipo primitivo, en realidad es un **objeto**, y tiene una particularidad muy importante: es **immutable**.

Inmutabilidad: la clave para entender String

Cuando "modificas" un String, en realidad Java crea uno nuevo en memoria; el original nunca cambia.

```
String saludo = "Hola";
saludo.concat(" mundo"); // esto NO modifica saludo
System.out.println(saludo); // sigue imprimiendo "Hola"

saludo = saludo.concat(" mundo"); // hay que reasignar
System.out.println(saludo); // ahora sí: "Hola mundo"
```

Métodos comunes de String

```
String texto = "  Java es divertido  ";

texto.length();           // 21 (longitud, incluye espacios)
texto.trim();              // "Java es divertido" (sin espacios extremos)
texto.toUpperCase();       // "  JAVA ES DIVERTIDO  "
texto.toLowerCase();       // "  java es divertido  "
texto.contains("Java");    // true
texto.replace("Java", "Python"); // reemplaza texto
texto.trim().split(" ");   // divide en un array: ["Java","es","divertido"]
texto.trim().charAt(0);    // 'J' (carácter en la posición 0)
texto.trim().substring(0, 4); // "Java" (subcadena)
```

Comparar cadenas: == vs equals()

Este es uno de los errores más comunes entre quienes empiezan en Java. `==` compara si dos variables apuntan al mismo objeto en memoria. `equals()` compara si el **contenido** del texto es igual.

```
String a = new String("hola");
String b = new String("hola");

System.out.println(a == b);           // false (objetos distintos en memoria)
System.out.println(a.equals(b));      // true  (mismo contenido)
```

■■ **CUIDADO:** Para comparar el contenido de dos Strings, usa siempre `.equals()`, nunca `==`. Esta regla aplica a casi todos los objetos, no solo a String.

StringBuilder: para construir texto eficientemente

Si necesitas concatenar texto muchas veces (por ejemplo, dentro de un bucle), usar + repetidamente es ineficiente porque crea un String nuevo cada vez. StringBuilder resuelve esto modificando un mismo objeto "mutable" en memoria.

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 3; i++) {
    sb.append("Línea ").append(i).append("\n");
}
System.out.println(sb.toString());
```

Capítulo 9. Métodos: organizando el código en bloques reutilizables

Un método es un bloque de código con nombre que realiza una tarea específica y puede reutilizarse cuantas veces sea necesario.

Anatomía de un método

```
public static int sumar(int a, int b) {  
    int resultado = a + b;  
    return resultado;  
}
```

- `public`: modificador de acceso (quién puede usarlo).
- `static`: pertenece a la clase, no a un objeto en particular (lo veremos a fondo en el capítulo de contexto).
- `int`: tipo de dato que **devuelve** el método.
- `sumar`: el nombre del método.
- `(int a, int b)`: los **parámetros** que recibe.
- `return`: entrega el resultado a quien llamó al método.

```
public class Calculadora {  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int total = sumar(5, 3);  
        System.out.println(total); // 8  
    }  
}
```

Métodos sin valor de retorno: void

Cuando un método solo realiza una acción, sin devolver ningún dato, se declara con `void`.

```
public static void saludar(String nombre) {  
    System.out.println("Hola, " + nombre + "!");  
}
```

Sobrecarga de métodos (overloading)

Java permite tener varios métodos con el mismo nombre, siempre que tengan una lista de parámetros diferente (distinta cantidad o distinto tipo).

```
public static int sumar(int a, int b) {  
    return a + b;  
}  
  
public static double sumar(double a, double b) {  
    return a + b;  
}  
  
public static int sumar(int a, int b, int c) {  
    return a + b + c;  
}
```

Paso de parámetros por valor

En Java, los parámetros siempre se pasan **por valor**: el método recibe una copia. Para tipos primitivos, eso significa que los cambios dentro del método no afectan a la variable original. Para objetos, se copia la referencia (la "dirección"), así que sí puedes modificar el contenido del objeto, pero no puedes hacer que la variable original apunte a otro objeto.

```
public static void incrementar(int numero) {  
    numero = numero + 1; // solo cambia la copia local  
}  
  
public static void main(String[] args) {  
    int valor = 5;  
    incrementar(valor);  
    System.out.println(valor); // sigue siendo 5  
}
```

Varargs: número variable de argumentos

```
public static int sumarTodo(int... numeros) {  
    int total = 0;  
    for (int n : numeros) total += n;  
    return total;  
}  
  
// Se puede llamar con cualquier cantidad de argumentos:  
sumarTodo(1, 2);  
sumarTodo(1, 2, 3, 4, 5);
```

Recursividad

Un método puede llamarse a sí mismo. Esto se llama recursividad, y es útil para problemas que se pueden dividir en subproblemas más pequeños.

```
public static int factorial(int n) {  
    if (n <= 1) return 1;           // caso base: detiene la recursión  
    return n * factorial(n - 1);    // caso recursivo  
}  
  
// factorial(5) = 5 * 4 * 3 * 2 * 1 = 120
```

■■ **CUIDADO:** Toda función recursiva necesita un **caso base** que detenga las llamadas; de lo contrario, el programa entrará en un ciclo infinito y terminará con un error de *StackOverflowError*.

Capítulo 10. Programación Orientada a Objetos (POO): clases y objetos

Hasta ahora todo nuestro código ha vivido dentro de un único método `main`. La Programación Orientada a Objetos (POO) propone algo distinto: modelar el programa como un conjunto de **objetos** que representan cosas del mundo real, cada uno con sus propios datos y comportamientos.

Clase vs objeto

Una **clase** es un molde o plantilla que define qué datos (*atributos*) y qué comportamientos (*métodos*) tendrán los objetos creados a partir de ella. Un **objeto** (o instancia) es una "materialización" concreta de esa clase, con valores propios.

```
public class Perro {
    // Atributos (el estado del objeto)
    String nombre;
    int edad;

    // Método (el comportamiento del objeto)
    void ladrar() {
        System.out.println(nombre + " dice: ¡Guau!");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Perro miPerro = new Perro(); // se crea un objeto (instancia)
        miPerro.nombre = "Rocky";
        miPerro.edad = 3;
        miPerro.ladrar();             // Rocky dice: ¡Guau!

        Perro otroPerro = new Perro(); // otro objeto, independiente
        otroPerro.nombre = "Luna";
        otroPerro.ladrar();           // Luna dice: ¡Guau!
    }
}
```

■ **NOTA:** Cada objeto tiene su propia copia de los atributos. `miPerro` y `otroPerro` son independientes: cambiar el nombre de uno no afecta al otro.

Constructores

Un constructor es un método especial que se ejecuta automáticamente al crear un objeto con `new`. Sirve para inicializar los atributos desde el principio, en lugar de asignarlos uno por uno después.


```
public class Perro {  
    String nombre;  
    int edad;  
  
    // Constructor: mismo nombre que la clase, sin tipo de retorno  
    public Perro(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    void ladrar() {  
        System.out.println(nombre + " dice: ¡Guau!");  
    }  
}
```

```
Perro rocky = new Perro("Rocky", 3); // se inicializa en una sola línea
```

La palabra clave this

Dentro de una clase, `this` hace referencia al objeto actual: "este mismo objeto sobre el que se está ejecutando el método". Se usa sobre todo cuando un parámetro tiene el mismo nombre que un atributo, para distinguir uno de otro (como en el constructor de arriba: `this.nombre` es el atributo, `nombre` a secas es el parámetro).

Constructor por defecto

Si no escribes ningún constructor, Java te da uno automáticamente, sin parámetros, que no inicializa nada en especial. En cuanto escribes tu propio constructor, ese constructor por defecto desaparece.

Capítulo 11. El contexto en Java: ámbito, this y static

Este capítulo reúne uno de los temas que más confusión genera al aprender Java: el **contexto** en el que vive cada variable y cada método. Entender el contexto te dice, en todo momento, *qué significa una palabra* (una variable, this, un método) según *dónde* está escrita.

11.1 Ámbito (scope) de las variables

El **ámbito** de una variable es la región de código donde esa variable existe y puede usarse. Fuera de ese contexto, la variable simplemente no existe para el compilador.

- **Variables locales:** declaradas dentro de un método o bloque. Solo existen mientras ese método se está ejecutando, y desaparecen al terminar.
- **Variables de instancia (o de objeto):** declaradas dentro de la clase, fuera de cualquier método. Cada objeto tiene su propia copia, y existen mientras el objeto exista.
- **Variables de clase (static):** declaradas con `static`. Pertenecen a la clase en sí, no a un objeto en particular; existe una sola copia compartida por todos los objetos.
- **Variables de bloque:** declaradas dentro de un `if`, `for` u otro bloque `{ }`. Su contexto termina en la llave de cierre de ese bloque.

```
public class Ejemplo {  
    static int contadorGlobal = 0;    // contexto de clase (static)  
    int valorPropio;                 // contexto de instancia  
  
    void metodo() {  
        int local = 10;              // contexto local al método  
        if (local > 5) {  
            int soloAqui = 99;        // contexto del bloque if  
            System.out.println(soloAqui);  
        }  
        // System.out.println(soloAqui); // ERROR: fuera de su contexto  
    }  
}
```

■■ **CUIDADO:** Una variable declarada dentro de un bloque `{ }` "muere" en cuanto ese bloque termina. Intentar usarla fuera produce un error de compilación: "cannot find symbol".

11.2 Contexto estático vs contexto de instancia

Esta es la distinción más importante del capítulo. Un miembro `static` vive en el **contexto de la clase**: existe una única copia, compartida, incluso antes de crear ningún objeto. Un miembro de instancia vive en el **contexto de cada objeto**: cada objeto tiene la suya propia.

```
public class Contador {
    static int totalContadores = 0; // compartido por TODOS los objetos
    int miValor = 0;                // propio de CADA objeto

    public Contador() {
        totalContadores++; // se incrementa la copia compartida
        miValor++;         // se incrementa solo la copia de este objeto
    }
}
```

```
Contador c1 = new Contador();
Contador c2 = new Contador();
Contador c3 = new Contador();

System.out.println(Contador.totalContadores); // 3 (compartido)
System.out.println(c1.miValor); // 1 (propio de c1)
System.out.println(c2.miValor); // 1 (propio de c2, no se contagia de c1)
```

La regla práctica más importante: **dentro de un método static, no existe un objeto "actual"**, por lo tanto no puedes usar `this` ni acceder directamente a atributos de instancia. Por eso, cuando empezaste con `public static void main`, solo podías llamar a otros métodos `static` sin crear un objeto primero.

```
public class Ejemplo {
    int valor = 10; // de instancia

    static void metodoEstatico() {
        // System.out.println(valor); // ERROR: no hay "this" en contexto static
        System.out.println("Este método no pertenece a ningún objeto");
    }

    void metodoDeInstancia() {
        System.out.println(valor); // OK: this.valor implícito
    }
}
```

11.3 El significado contextual de `this`

`this` siempre se refiere al objeto sobre el que se está ejecutando el método en ese momento. Su significado cambia según el contexto de ejecución: dentro de un método de `c1`, `this` es `c1`; dentro de un método de `c2`, `this` es `c2`. Por eso no puede existir dentro de un contexto `static`: allí no hay un objeto concreto al cual referirse.

11.4 Contexto en clases anidadas y lambdas

Cuando defines una clase dentro de otra, o una expresión lambda dentro de un método, el contexto se anida: el código interno puede "ver" las variables del contexto externo, siempre que sean efectivamente finales (no cambian después de asignarse).

```
public class Reloj {  
    void iniciar() {  
        int minutos = 5; // variable del contexto externo (el método)  
  
        Runnable tarea = () -> {  
            // la lambda "captura" el contexto de iniciar()  
            System.out.println("Faltan " + minutos + " minutos");  
        };  
        tarea.run();  
    }  
}
```

■ **CONSEJO:** Fuera del lenguaje básico, la palabra "contexto" también aparece en frameworks como Spring, donde un *ApplicationContext* es el objeto central que crea y administra todos los componentes de una aplicación. La idea es la misma en el fondo: un contexto es "el entorno en el que algo vive y del que puede tomar lo que necesita".

Capítulo 12. Encapsulamiento y modificadores de acceso

El encapsulamiento es el principio de POO que consiste en **ocultar los detalles internos** de un objeto y exponer solo lo necesario a través de métodos controlados. Esto evita que otras partes del programa dejen un objeto en un estado inválido.

Modificadores de acceso

- `private`: solo accesible dentro de la misma clase.
- (sin modificador / "package-private"): accesible dentro del mismo paquete.
- `protected`: accesible dentro del paquete y por clases que hereden de esta (lo veremos en el capítulo de herencia).
- `public`: accesible desde cualquier parte del programa.

Getters y setters

La práctica habitual es declarar los atributos como `private` y exponer métodos públicos (*getters* para leer, *setters* para modificar) que pueden incluir validaciones.

```
public class CuentaBancaria {
    private double saldo;

    public double getSaldo() {
        return saldo;
    }

    public void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
        } else {
            System.out.println("El monto debe ser positivo");
        }
    }
}
```

```
CuentaBancaria cuenta = new CuentaBancaria();
cuenta.depositar(100);
// cuenta.saldo = -500; // ERROR: saldo es privado, no accesible desde afuera
System.out.println(cuenta.getSaldo()); // 100.0
```

■ **CONSEJO:** Gracias al encapsulamiento, nadie puede poner el saldo en negativo "a la fuerza" desde fuera de la clase: toda modificación pasa por el método `depositar`, que valida la operación.

Capítulo 13. Herencia

La herencia permite que una clase (subclase) reutilice atributos y métodos de otra clase (superclase), evitando repetir código y modelando relaciones del tipo "es un/una" (un Perro es *un* Animal).

```
public class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void comer() {
        System.out.println(nombre + " está comiendo");
    }
}
```

```
public class Perro extends Animal {
    public Perro(String nombre) {
        super(nombre); // llama al constructor de Animal
    }

    public void ladrar() {
        System.out.println(nombre + " dice: ¡Guau!");
    }
}
```

```
Perro rocky = new Perro("Rocky");
rocky.comer(); // heredado de Animal: "Rocky está comiendo"
rocky.ladrar(); // propio de Perro: "Rocky dice: ¡Guau!"
```

La palabra clave super

`super` hace referencia a la superclase. Se usa para llamar a su constructor (como arriba) o para invocar una versión sobrescrita de un método.

Sobrescritura de métodos (override)

Una subclase puede redefinir el comportamiento de un método heredado usando la anotación `@Override`, que le pide al compilador que verifique que realmente estás sobrescribiendo un método existente.

```
public class Perro extends Animal {  
    public Perro(String nombre) { super(nombre); }  
  
    @Override  
    public void comer() {  
        System.out.println(nombre + " come croquetas felizmente");  
    }  
}
```

■ **NOTA:** Una clase puede marcarse como `final` para impedir que otras clases hereden de ella, y un método puede marcarse `final` para impedir que se sobrescriba.

Capítulo 14. Polimorfismo

"Polimorfismo" significa "muchas formas". En Java se manifiesta principalmente de dos maneras: la sobrecarga de métodos (ya vista) y la capacidad de tratar objetos de distintas subclases a través de una referencia de la superclase, ejecutando en cada caso el comportamiento específico de cada subclase.

```
public class Gato extends Animal {  
    public Gato(String nombre) { super(nombre); }  
  
    @Override  
    public void comer() {  
        System.out.println(nombre + " come pescado con cuidado");  
    }  
}
```

```
Animal[] animales = { new Perro("Rocky"), new Gato("Michi") };  
  
for (Animal a : animales) {  
    a.comer(); // cada uno ejecuta SU propia versión de comer()  
}  
// Rocky come croquetas felizmente  
// Michi come pescado con cuidado
```

Esto es polimorfismo en acción: el mismo código (`a.comer()`) produce comportamientos diferentes según el tipo real del objeto en tiempo de ejecución.

Upcasting, downcasting e instanceof

```
Animal miAnimal = new Perro("Rocky"); // upcasting implícito  
  
if (miAnimal instanceof Perro) {  
    Perro p = (Perro) miAnimal; // downcasting explícito  
    p.ladRAR();  
}  
  
// Forma moderna (Java 16+), con "pattern matching":  
if (miAnimal instanceof Perro p) {  
    p.ladRAR();  
}
```


Capítulo 15. Clases abstractas e interfaces

Clases abstractas

Una clase abstracta no puede instanciarse directamente (no puedes hacer new de ella); su propósito es servir de base común para otras clases, y puede declarar métodos **abstractos** (sin cuerpo) que las subclases están obligadas a implementar.

```
public abstract class Figura {  
    public abstract double calcularArea(); // sin cuerpo  
  
    public void describir() {  
        System.out.println("Mi área es: " + calcularArea());  
    }  
}
```

```
public class Circulo extends Figura {  
    private double radio;  
    public Circulo(double radio) { this.radio = radio; }  
  
    @Override  
    public double calcularArea() {  
        return Math.PI * radio * radio;  
    }  
}
```

Interfaces

Una interfaz define un "contrato": una lista de métodos que cualquier clase que la implemente debe proporcionar. A diferencia de las clases abstractas, una clase puede implementar **varias** interfaces a la vez, aunque solo puede heredar de una única clase.

```
public interface Volador {  
    void volar();  
}  
  
public interface Nadador {  
    void nadar();  
}  
  
public class Pato implements Volador, Nadador {  
    @Override  
    public void volar() { System.out.println("El pato vuela bajito"); }  
  
    @Override  
    public void nadar() { System.out.println("El pato nada en el lago"); }  
}
```

Métodos default y static en interfaces

Desde Java 8, una interfaz también puede incluir métodos con cuerpo (default), que las clases heredan automáticamente si no los sobrescriben.

```
public interface Volador {  
    void volar();  
  
    default void aterrizar() {  
        System.out.println("Aterrizando suavemente...");  
    }  
}
```

■ **CONSEJO:** Clase abstracta vs interfaz, en una frase: usa una clase abstracta cuando las clases comparten **estado** (atributos) y comportamiento común; usa una interfaz cuando solo necesitas garantizar que ciertas clases, aunque no estén relacionadas entre sí, **puedan hacer lo mismo** (por ejemplo, "volar").

Capítulo 16. Manejo de excepciones

Una excepción es un evento que interrumpe el flujo normal del programa cuando ocurre un error (dividir entre cero, acceder a un índice inexistente, un archivo que no existe, etc.). Java permite "capturar" esos errores y decidir qué hacer, en lugar de dejar que el programa se caiga de golpe.

try / catch / finally

```
try {
    int resultado = 10 / 0; // lanza ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Error: no se puede dividir entre cero");
} finally {
    System.out.println("Este bloque se ejecuta siempre, haya o no error");
}
```

- **try:** el bloque de código que puede fallar.
- **catch:** captura un tipo específico de excepción y define qué hacer si ocurre.
- **finally:** se ejecuta siempre, haya habido error o no (ideal para cerrar recursos como archivos o conexiones).

Excepciones checked vs unchecked

- **Checked (verificadas):** el compilador te obliga a manejarlas con try/catch o a declararlas con throws. Ejemplo: `IOException`.
- **Unchecked (no verificadas):** ocurren en tiempo de ejecución y no es obligatorio capturarlas, aunque a menudo conviene hacerlo. Ejemplo: `NullPointerException`, `ArithmeticException`.

try-with-resources

Cuando trabajas con recursos que deben cerrarse (archivos, conexiones), esta forma de try garantiza el cierre automático, incluso si ocurre un error.

```
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea = br.readLine();
    System.out.println(linea);
} catch (IOException e) {
    System.out.println("No se pudo leer el archivo: " + e.getMessage());
}
```

Crear excepciones propias

```
public class SaldoInsuficienteException extends Exception {  
    public SaldoInsuficienteException(String mensaje) {  
        super(mensaje);  
    }  
}
```

```
public void retirar(double monto) throws SaldoInsuficienteException {  
    if (monto > saldo) {  
        throw new SaldoInsuficienteException("Saldo insuficiente para retirar " + monto);  
    }  
    saldo -= monto;  
}
```

■ **NOTA:** Cada bloque catch se ejecuta en su propio contexto de manejo de errores: dentro de él, la variable de la excepción (por ejemplo e) solo existe en ese bloque, igual que cualquier otra variable local.

Capítulo 17. Colecciones: listas, conjuntos y mapas

Los arrays tienen un tamaño fijo. El paquete `java.util` ofrece **colecciones** que crecen y se encogen dinámicamente, con muchas utilidades incorporadas.

ArrayList: una lista dinámica

```
import java.util.ArrayList;
import java.util.List;

List<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Eva");
nombres.remove("Luis");

for (String nombre : nombres) {
    System.out.println(nombre);
}
System.out.println("Total: " + nombres.size());
```

HashSet: elementos sin duplicados

```
import java.util.HashSet;
import java.util.Set;

Set<String> colores = new HashSet<>();
colores.add("rojo");
colores.add("azul");
colores.add("rojo"); // se ignora, ya existe

System.out.println(colores.size()); // 2
```

HashMap: pares clave-valor

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 25);
edades.put("Luis", 30);

System.out.println(edades.get("Ana")); // 25

for (Map.Entry<String, Integer> entrada : edades.entrySet()) {
    System.out.println(entrada.getKey() + " tiene " + entrada.getValue() + " años");
}
```

■ **CONSEJO:** ¿Cuándo usar cada una? `List` cuando el orden importa y puede haber repetidos; `Set` cuando no quieres duplicados; `Map` cuando necesitas buscar valores rápidamente por una clave (como un diccionario).

Capítulo 18. Genéricos

Los genéricos permiten escribir clases y métodos que funcionan con cualquier tipo de dato, manteniendo la seguridad de tipos del compilador. Ya los usaste sin saberlo: `List<String>` es una lista **genérica** especializada en `String`.

Una clase genérica propia

```
public class Caja<T> {  
    private T contenido;  
  
    public void guardar(T contenido) {  
        this.contenido = contenido;  
    }  
  
    public T obtener() {  
        return contenido;  
    }  
}
```

```
Caja<String> cajaTexto = new Caja<>();  
cajaTexto.guardar("Hola");  
String valor = cajaTexto.obtener(); // no hace falta casting  
  
Caja<Integer> cajaNumero = new Caja<>();  
cajaNumero.guardar(42);
```

Métodos genéricos

```
public static <T> void imprimirArray(T[] array) {  
    for (T elemento : array) {  
        System.out.println(elemento);  
    }  
}
```

■ **NOTA:** El nombre `T` es solo una convención (viene de "Type"); podrías llamarlo de otra forma. Otras convenciones comunes son `K` y `V` para clave y valor en mapas.

Capítulo 19. Programación funcional: lambdas y Streams

Desde Java 8, es posible tratar el *comportamiento* (no solo los datos) como algo que se puede pasar de un lado a otro, gracias a las expresiones lambda.

Interfaces funcionales

Una interfaz funcional es una interfaz con un único método abstracto. Java incluye varias predefinidas en `java.util.function`.

```
interface Operacion {  
    int aplicar(int a, int b);  
}
```

Expresiones lambda

Una lambda es una forma corta de escribir la implementación de una interfaz funcional, sin necesidad de crear una clase completa.

```
Operacion suma = (a, b) -> a + b;  
Operacion resta = (a, b) -> a - b;  
  
System.out.println(suma.aplicar(5, 3)); // 8  
System.out.println(resta.aplicar(5, 3)); // 2
```

```
import java.util.function.Function;  
  
Function<Integer, Integer> alCuadrado = n -> n * n;  
System.out.println(alCuadrado.apply(5)); // 25
```

Streams: procesar colecciones de forma declarativa

La API de Streams permite transformar y filtrar colecciones encadenando operaciones, en lugar de escribir bucles manuales.


```
import java.util.List;
import java.util.stream.Collectors;

List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

List<Integer> pares = numeros.stream()
    .filter(n -> n % 2 == 0)      // se queda solo con los pares
    .map(n -> n * n)             // eleva cada uno al cuadrado
    .collect(Collectors.toList()); // vuelve a convertir en lista

System.out.println(pares); // [4, 16, 36, 64, 100]
```

```
int suma = numeros.stream()
    .mapToInt(Integer::intValue)
    .sum();
System.out.println(suma); // 55
```

■ **CONSEJO:** Los streams no modifican la colección original: cada operación produce un nuevo stream, y la colección de origen permanece intacta.

Capítulo 20. Entrada/salida (I/O) y archivos

Leer datos desde el teclado

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("¿Cómo te llamas? ");
String nombre = sc.nextLine();
System.out.print("¿Cuántos años tienes? ");
int edad = sc.nextInt();

System.out.println("Hola " + nombre + ", tienes " + edad + " años.");
```

Escribir y leer archivos de texto

```
import java.io.FileWriter;
import java.io.IOException;

try (FileWriter fw = new FileWriter("notas.txt")) {
    fw.write("Primera línea\n");
    fw.write("Segunda línea\n");
} catch (IOException e) {
    System.out.println("Error al escribir: " + e.getMessage());
}
```

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

try (BufferedReader br = new BufferedReader(new FileReader("notas.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.out.println("Error al leer: " + e.getMessage());
}
```

Capítulo 21. Concurrency básica: hilos (Threads)

Un hilo (*thread*) es una línea independiente de ejecución dentro de un programa. Permite que varias tareas avancen "al mismo tiempo" (o de forma intercalada, según el número de núcleos disponibles).

Crear un hilo con Runnable

```
Runnable tarea = () -> {
    for (int i = 1; i <= 3; i++) {
        System.out.println("Trabajando... " + i);
    }
};

Thread hilo = new Thread(tarea);
hilo.start(); // inicia el hilo (no llames a run() directamente)
```

■■ **CUIDADO:** Llamar a `hilo.run()` en lugar de `hilo.start()` es un error muy común: eso simplemente ejecuta el código en el hilo actual, sin crear uno nuevo.

El contexto de un hilo

Cada hilo tiene su propio contexto de ejecución: su propia pila de llamadas y sus propias variables locales. Sin embargo, todos los hilos de un mismo programa comparten el mismo contexto de objetos y variables static, por lo que modificar un dato compartido desde varios hilos a la vez puede causar resultados inesperados si no se sincroniza.

```
public class Contador {
    private int valor = 0;

    public synchronized void incrementar() {
        valor++; // "synchronized" evita que dos hilos lo modifiquen a la vez
    }

    public int getValor() {
        return valor;
    }
}
```

Capítulo 22. Buenas prácticas y hacia dónde seguir

Convenciones de nombres

- Clases: PascalCase — Ej. CuentaBancaria.
- Métodos y variables: camelCase — Ej. calcularTotal, saldoActual.
- Constantes: MAYUSCULAS_CON_GUIONES_BAJOS — Ej. IVA_PORCENTAJE.
- Paquetes: minúsculas, sin espacios — Ej. com.miempresa.app.

Javadoc: documentar tu código

```
/**
 * Calcula el área de un círculo.
 *
 * @param radio el radio del círculo, debe ser positivo
 * @return el área calculada
 */
public double calcularArea(double radio) {
    return Math.PI * radio * radio;
}
```

Pruebas automatizadas con JUnit (introducción)

A medida que tus programas crecen, conviene escribir pruebas que verifiquen automáticamente que tu código sigue funcionando correctamente. JUnit es la biblioteca más usada en el ecosistema Java para esto.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculadoraTest {
    @Test
    void sumaDeDosNumeros() {
        Calculadora calc = new Calculadora();
        assertEquals(8, calc.sumar(5, 3));
    }
}
```

Herramientas de construcción: Maven y Gradle

Los proyectos Java reales rara vez se compilan a mano con `javac`. Se usan herramientas como **Maven** o **Gradle**, que gestionan automáticamente las dependencias (bibliotecas externas), la compilación, las pruebas y el empaquetado del proyecto.

¿Qué seguir aprendiendo?

- 1 **Spring / Spring Boot:** el framework más usado para construir aplicaciones y APIs web en Java. Aquí conocerás a fondo el concepto de "ApplicationContext" que mencionamos en el capítulo 11.
- 2 **Bases de datos con JDBC / JPA / Hibernate:** para que tus programas guarden y consulten información de forma persistente.
- 3 **Control de versiones con Git:** imprescindible para trabajar en equipo y llevar un historial ordenado de tu código.
- 4 **Patrones de diseño:** soluciones probadas a problemas comunes de diseño de software (Singleton, Factory, Observer, entre otros).
- 5 **Estructuras de datos y algoritmos:** para escribir programas más eficientes y prepararte para entrevistas técnicas.

■ **CONSEJO:** El aprendizaje de un lenguaje nunca termina realmente: la mejor forma de consolidar todo lo visto en esta guía es escribir proyectos propios, por pequeños que sean. Empieza con algo simple —una lista de tareas, una calculadora, un pequeño juego de adivinar números— y ve incorporando cada concepto que aprendiste aquí.